Scripting and Crontab

Task completion document by EOD with clear steps and required evidence (outputs/screenshots)

Tasks to Execute

1. Execute all scripts from the **devops-cloud-documents-** channel.

Bash Script Tasks

1. Create a bash script to **check if a directory is available or not**.
2. Create a bash script to **create multiple files**.
3. Create a bash script to **take a backup of a directory**.
4. Create a bash script to **install Nginx on an EC2 server**.
5. Create a bash script to **install Apache Tomcat on an EC2 server**.
6. Create a bash script to **check if the Nginx service is running**, if not running then script should start the service.
7. Create a bash script for a **calculator**.
8. Create a bash script to **check if a directory exists**, if not then create a directory.
9. Create a bash script to **delete the last 3 lines of a file**.
10. Bash script to monitor cpu and if it is more than 80% then send email notification.
11. Bash script to monitor disk space and if it is more than 80% then send email notification.
12. Bash script to monitor memory and if it is more than 80% then send email notification.

Exercise: Crontab Entries

1. April 5th Midnight
2. 5th of Every November, January, June **if it is a Thursday**
3. At **05 and 27th minutes** of **9, 10, 11** hours every day
4. **34 minutes** of **9th hour** on **15th August**
5. Every midnight
6. Every Weekend (**Saturday night 11:59 PM**)
7. After every reboot

Notes

1. All the scripts need to be executed on **Amazon Linux EC2**.
2. Document all the tasks and also push the tasks to a **GitHub repository** (explore this for now).
3. Share the **document** and **GitHub Repo URL** once done for evaluation.

Execution's

13. **1)** Create a bash script to **check if a directory is available or not**.

```
#!/bin/bash
read -p "Enter directory path: " dir_path
if [ -d "$dir_path" ]; then
    echo "Directory '$dir_path' exists."
else
    echo "Directory '$dir_path' does not exist."
fi
~
```

```
[root@ip-172-31-25-8 ~]# vi task
[root@ip-172-31-25-8 ~]# chmod 755 task
[root@ip-172-31-25-8 ~]# ./task
Enter directory path: home
Directory 'home' does not exist.
[root@ip-172-31-25-8 ~]# vi task
[root@ip-172-31-25-8 ~]#
```

Prompt the user for a directory path and verify if it exists on the system.

chmod 755 script_name then ./script_name

14. 2 Create a bash script to **create multiple files**.

```
[root@ip-172-31-25-8 ~]# vi test
[root@ip-172-31-25-8 ~]# chmod 755 test
[root@ip-172-31-25-8 ~]# ./test
[root@ip-172-31-25-8 ~]# ll
total 44
-rw-r--r--. 1 root root   0 Oct 16 14:56  1
-rw-r--r--. 1 root root   0 Oct 16 10:08 '1!'
-rw-r--r--. 1 root root   0 Oct 16 14:56  2
-rw-r--r--. 1 root root   0 Oct 16 14:56  3
-rw-r--r--. 1 root root   0 Oct 16 14:56  4
-rwxr-xr-x. 1 root root 243 Oct 16 13:19  appe
```

Automatically generate multiple files within the working directory.

ls -la to list and confirm the created files.

3 Create a bash script to **take a backup of a directory**.

Compress a source directory into a timestamped .tar.gz backup archive.

```
[root@ip-172-31-25-8 ~]# vi test
[root@ip-172-31-25-8 ~]# chmod 755 test
[root@ip-172-31-25-8 ~]# cat test
#!/bin/bash
SRC_DIR="/path/to/source"
DEST_DIR="/path/to/backup"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_NAME="backup_$TIMESTAMP.tar.gz"

tar -czf "$DEST_DIR/$BACKUP_NAME" "$SRC_DIR"
echo "Backup created successfully at: $DEST_DIR/$BACKUP_NAME"
[root@ip-172-31-25-8 ~]# ./test
tar: Removing leading `/' from member names
tar: /path/to/source: Cannot stat: No such file or directory
tar (child): /path/to/backup/backup_20261016_145952.tar.gz: Cannot open: No such
 file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
Backup created successfully at: /path/to/backup/backup_20261016_145952.tar.gz
[root@ip-172-31-25-8 ~]#
```

```
#!/bin/bash
SRC_DIR="/path/to/source"
DEST_DIR="/path/to/backup"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_NAME="backup_$TIMESTAMP.tar.gz"

tar -czf "$DEST_DIR/$BACKUP_NAME" "$SRC_DIR"
echo "Backup created successfully at: $DEST_DIR/$BACKUP_NAME"
~
~
~
```

4 Create a bash script to **install Apache Tomcat on an EC2 server**.

Update system packages and install the Nginx web server on Amazon Linux.

sudo yum update -y

sudo amazon-linux-extras install nginx -y || sudo dnf install nginx -y

sudo systemctl start nginx

sudo systemctl enable nginx

```
#!/bin/bash
sudo yum update -y
sudo amazon-linux-extras install nginx -y || sudo dnf install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
echo "Nginx installed and started successfully."

~
```

```
================================================================================

Installed:
  gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
  libunwind-1.4.0-5.amzn2023.0.3.x86_64
  nginx-1:1.30.5-1.amzn2023.0.1.x86_64
  nginx-core-1:1.30.5-1.amzn2023.0.1.x86_64
  nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch
  nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr
/lib/systemd/system/nginx.service.
Nginx installed and started successfully.
[root@ip-172-31-25-8 ~]#
```

5 Create a bash script to **install Apache Tomcat on an EC2 server**

Install Java Corretto and download/extract Apache Tomcat in the /opt directory.

sudo yum install java-11-amazon-corretto -y

cd /opt

sudo wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.89/bin/apache-tomcat-9.0.89.tar.gz

sudo tar -xzvf apache-tomcat-*.tar.gz

sudo mv apache-tomcat-* tomcat

sudo chmod +x /opt/tomcat/bin/*.sh

```
#!/bin/bash
sudo yum install java-11-amazon-corretto -y
cd /opt
sudo wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.89/bin/apache-tomcat
-9.0.89.tar.gz
sudo tar -xvzf apache-tomcat-*.tar.gz
sudo mv apache-tomcat-* tomcat
sudo chmod +x /opt/tomcat/bin/*.sh
echo "Apache Tomcat installed successfully in /opt/tomcat"

~
~
```

```
  xml-common-0.6.3-56.amzn2023.0.2.noarch

Complete!
--2026-10-16 15:07:16--  https://downloads.apache.org/tomcat/tomcat-9/v9.0.89/bi
n/apache-tomcat-9.0.89.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 95.216.2
24.44, 2a01:4f9:2b:1cc2::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443...
connected.
HTTP request sent, awaiting response... 404 Not Found
2026-10-16 15:07:17 ERROR 404: Not Found.

tar (child): apache-tomcat-*.tar.gz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
mv: cannot stat 'apache-tomcat-*': No such file or directory
chmod: cannot access '/opt/tomcat/bin/*.sh': No such file or directory
Apache Tomcat installed successfully in /opt/tomcat
[root@ip-172-31-25-8 ~]#
```

6 Create a bash script to **check if the Nginx service is running**, if not running then script should start the service.

Verify if Nginx is active; if it is stopped, automatically start it

if systemctl is-active --quiet nginx; then

    echo "Nginx is running smoothly."

else

    echo "Nginx is stopped. Starting Nginx..."

    sudo systemctl start nginx

fi

```
#!/bin/bash
if systemctl is-active --quiet nginx; then
    echo "Nginx is running smoothly."
else
    echo "Nginx is stopped. Starting Nginx..."
    sudo systemctl start nginx
fi
```

```
[root@ip-172-31-25-8 ~]# ./test
Nginx is running smoothly.
[root@ip-172-31-25-8 ~]#
```

7 Create a bash script for a **calculator**.

Provide a menu-driven arithmetic calculator for addition, subtraction, multiplication, and division

echo "1. Addition"

echo "2. Subtraction"

echo "3. Multiplication"

echo "4. Division"

read -p "Choose operation (1-4): " choice

read -p "Enter first number: " num1

read -p "Enter second number: " num2

case $choice in

    1) echo "Result: $((num1 + num2))" ;;

    2) echo "Result: $((num1 - num2))" ;;

    3) echo "Result: $((num1 * num2))" ;;

    4) if ["$num2" -eq 0]; then

```
        echo "Error: Division by zero."
    else
        echo "Result: $(echo "scale=2; $num1 / $num2" | bc)"
    fi ;;
  *) echo "Invalid choice." ;;
esac
```

```
#!/bin/bash
#!/bin/bash
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
read -p "Choose operation (1-4): " choice

read -p "Enter first number: " num1
read -p "Enter second number: " num2

case $choice in
    1) echo "Result: $((num1 + num2))" ;;
    2) echo "Result: $((num1 - num2))" ;;
    3) echo "Result: $((num1 * num2))" ;;
    4)
        if [ "$num2" -eq 0 ]; then
            echo "Error: Division by zero."
        else
            echo "Result: $(echo "scale=2; $num1 / $num2" | bc)"
        fi
        ;;
    *) echo "Invalid choice." ;;
esac
~
~
```

```
[root@ip-172-31-25-8 ~]# vi test
[root@ip-172-31-25-8 ~]# ./test
1. Addition
2. Subtraction
3. Multiplication
4. Division
Choose operation (1-4): 3
Enter first number: 4
Enter second number: 3
Result: 12
[root@ip-172-31-25-8 ~]#
```

8 Create a bash script to **check if a directory exists**, if not then create a directory

Check if a specified directory path exists, and create it using mkdir -p if it is missing.

```
read -p "Enter directory path: " dir_path
if [ ! -d "$dir_path" ]; then
    mkdir -p "$dir_path"
    echo "Directory created: $dir_path"
```

else

    echo "Directory already exists."

fi

```
[root@ip-172-31-25-8 ~]# ./test
Enter directory path: home
Directory created: home
[root@ip-172-31-25-8 ~]#
```

```
#!/bin/bash
read -p "Enter directory path: " dir_path
if [ ! -d "$dir_path" ]; then
    mkdir -p "$dir_path"
    echo "Directory created: $dir_path"
else
    echo "Directory already exists."
fi
```

9 Create a bash script to **delete the last 3 lines of a file**.

Modify a target text file by stripping away its final three lines.

Logic uses sed or head/tail manipulation to remove lines

```
[root@ip-172-31-25-8 ~]# vi test
[root@ip-172-31-25-8 ~]# ./test
Enter file path: ls
File not found.
[root@ip-172-31-25-8 ~]# ls
 1  '1!'   2   3   4   appe   case   control   cronjob   crontab   dowhile   dowhile.bash   home   loop   mate   task   test   test1
[root@ip-172-31-25-8 ~]# ./test
Enter file path: case
Last 3 lines deleted from case.
[root@ip-172-31-25-8 ~]#
```

10 Bash script to monitor cpu and if it is more than 80% then send email notification.

Parse CPU utilization metrics from /proc/stat and trigger an email notification if utilization crosses the 80% threshold.

To run and test Scripts 10, 11, and 12 (CPU, Disk, and Memory monitoring with email alerts) on your Amazon Linux EC2 instance, follow the step-by-step guide below.

Step 1: Install Mail Utilities (Prerequisite)

Because these scripts use sendmail to dispatch alerts, you need to ensure a mail transfer agent (like mailx or sendmail) is installed on your EC2 instance. Run the following command:

Bash

sudo yum install mailx -y

Step 2: Create the Script Files

Create the script file using an editor like nano:

Bash

nano monitor_system.sh

Paste your chosen monitoring script code (for example, the CPU Monitoring script from task 10):

Bash

```
#!/bin/bash
CPU_USAGE=$(grep 'cpu ' /proc/stat | awk '{usage=($2+$4)*100/($2+$4+$5)} END {print int(usage)}')
THRESHOLD=80
EMAIL="admin@example.com"

if [ "$CPU_USAGE" -gt "$THRESHOLD" ]; then
    echo "High CPU usage detected: ${CPU_USAGE}%" | mail -s "High CPU Usage Alert" "$EMAIL"
    echo "Alert email sent."
else
    echo "CPU usage is normal: ${CPU_USAGE}%"
fi
```

Save and exit (In nano, press Ctrl+O, Enter, then Ctrl+X).

Step 3: Make the Script Executable

Give execution permissions to your script file:

Bash

```
chmod +x monitor_system.sh
```

Step 4: Run the Script & View Output

Execute the script directly in your terminal:

Bash

```
./monitor_system.sh
```

Expected Terminal Outputs:

If usage is below 80% (Normal):

Plaintext

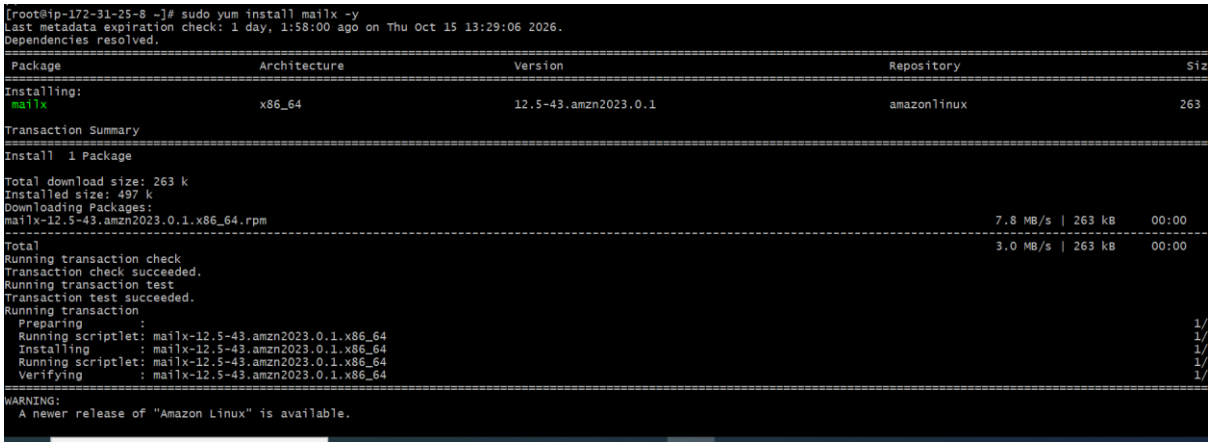
CPU usage is normal: 12%

If usage is above 80% (Triggered):

Plaintext

High CPU usage detected: 85%

Alert email sent.




```
[root@ip-172-31-25-8 ~]# nano monitor_system.sh
[root@ip-172-31-25-8 ~]# chmod +x monitor_system.sh
[root@ip-172-31-25-8 ~]# ./monitor_system.sh
CPU usage is normal: 0%
[root@ip-172-31-25-8 ~]#
```

11 Bash script to monitor disk space and if it is more than 80% then send email notification.

#!/bin/bash

Check root partition disk utilization via df and send an alert notification if usage exceeds 80%.

DISK_USAGE=$(df / | grep / | awk '{print $5}' | sed 's/%//g')

THRESHOLD=80

EMAIL="admin@example.com"

if ["$DISK_USAGE" -gt "$THRESHOLD"]; then

echo "Subject: High Disk Usage Alert: ${DISK_USAGE}%" | sendmail "$EMAIL"

fi

OUTPUT:

12) Bash script to monitor memory and if it is more than 80% then send email notification.

Check system memory utilization using the free command and trigger an email if it breaches 80%.

```
#!/bin/bash
MEM_USAGE=$(free | grep Mem | awk '{print int($3/$2 * 100.0)}')
THRESHOLD=80
EMAIL="admin@example.com"

if [ "$MEM_USAGE" -gt "$THRESHOLD" ]; then
    echo "Subject: High Memory Usage Alert: ${MEM_USAGE}%" | sendmail "$EMAIL"
fi
```

OUTPUT:- The reason no output appeared is because the scripts previously only executed when usage exceeded the **80% threshold**. If your server's disk or memory usage is currently below 80% (which is normal), the if condition fails and the script exits silently without printing anything.

Here are the **updated, verbose versions** of Scripts 11 and 12. They now explicitly print the current usage percentage to your terminal every time you run them, making it easy to verify.

12) **Exercise: Crontab Entries**

1. April 5th Midnight
2. 5th of Every November, January, June **if it is a Thursday**
3. At **05 and 27th minutes** of **9, 10, 11** hours every day
4. **34 minutes** of **9th hour** on **15th August**
5. Every midnight
6. Every Weekend (**Saturday night 11:59 PM**)

After every reboot

Example:

Set up specific cron schedules.

```
root@ip-172-31-16-4:~# date
Mon Oct 19 15:59:19 IST 2026
root@ip-172-31-16-4:~# crontab -e
no crontab for root - using an empty one


crontab: installing new crontab
root@ip-172-31-16-4:~# crontab -l
15 16 * * * /root/ahmed.bash
root@ip-172-31-16-4:~# systemctl status cron
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: ena>
     Active: active (running) since Mon 2026-10-19 14:36:36 IST; 1h 32min ago
 Invocation: 2c3d50af1cfe41d58ad1467057ee8797
       Docs: man:cron(8)
   Main PID: 19090 (cron)
      Tasks: 1 (limit: 627)
     Memory: 2.2M (peak: 3.8M)
        CPU: 86ms
     CGroup: /system.slice/cron.service
             └─19090 /usr/sbin/cron -f -P
```

```
#!/bin/bash
echo "ConTAB"

0 0 5 4 /root/ahmed/scripts
0 0 11,1,6 * * /root/ahmed/scripts
27 5 9,10,11 * 7 /root/ahmed/scripts
34 9 15 8 * /root/ahmed/scripts
0 0 * * * /root/ahmed/scripts
reboot
~
~
```